04. Feasify Cross-Module Integration Test Report

Document ID: FEASIFY-DOC-04

Scope: End-to-End Workflows, Real-Time Sync, and Cross-Role Consistency

Execution Date: November 11, 2026

Status: Completed & Verified

---

1. Integration Testing Objectives

Integration testing verifies that distributed modules across the Feasify ecosystem (Client Portals, Cloud Firestore listeners, Express 5 REST APIs, Socket.IO channels, and external government/geocoding services) exchange data reliably without data corruption, race conditions, or state desynchronization.

---

2. End-to-End Scenario Evaluations

Scenario 1: Student Proposal Submission to Adviser Live View

- **Flow:** Student logs in $\rightarrow$ Completes proposal form $\rightarrow$ Passes validation $\rightarrow$ Submits proposal $\rightarrow$ Document written to `proposals` collection $\rightarrow$ Group status updated to "Pending Review" $\rightarrow$ Notification dispatched $\rightarrow$ Adviser dashboard updates in real time.
- **Observed Behavior:** The Firestore write triggers the real-time `onSnapshot` listener in `AdviserDashboard.tsx`. The proposal card appears in the adviser's review queue with badge "Submitted" without requiring manual page reload.
- **Result: PASSED**

Scenario 2: Proposal Limit Enforcement Lifecycle

- **Flow:** Student creates Proposal 1 (Approved), Proposal 2 (Revision Required), and Proposal 3 (Draft) $\rightarrow$ Student attempts Proposal 4 $\rightarrow$ System queries `/api/teams/:groupId/proposals/count` $\rightarrow$ Allowed flag returns `false` $\rightarrow$ UI presents `SubmissionFailureModal` indicating maximum 3 proposals $\rightarrow$ Database remains unaltered.
- **Observed Behavior:** Existing proposals 1, 2, and 3 are preserved completely. No 4th record is inserted into Firestore.
- **Result: PASSED**

Scenario 3: Adviser Evaluation and Real-Time Student Feedback

- **Flow:** Adviser reviews submitted proposal $\rightarrow$ Selects "Revision Required" with detailed feedback comments $\rightarrow$ Dispatches `PUT /api/proposals/:proposalId/status` $\rightarrow$ Updates proposal status and appends feedback history $\rightarrow$ Student portal listener receives status update $\rightarrow$ Badge updates to "Needs Revision" $\rightarrow$ Adviser remarks box renders feedback.

- **Observed Behavior:** Status transition is instantaneous across browser windows via `onSnapshot(collection(db, "proposals"), where("groupId", "==", userGroup.id))`.
- **Result: PASSED**

Scenario 4: Chairperson Cross-Section Oversight

- **Flow:** Adviser approves proposal $\rightarrow$ Chairperson navigates to `/admin/projects` (`ChairpersonFeasib.tsx`) $\rightarrow$ Real-time listener aggregates groups and proposals across all academic sections $\rightarrow$ Analytics cards update approved count and section breakdown.
- **Observed Behavior:** Chairperson view successfully aggregates and displays records from Section A, B, and C.
- **Performance Caveat:** `ChairpersonFeasib.tsx` loads entire collections (`groups` , `proposals` , `users`) without pagination, causing heavy memory overhead when hundreds of records exist (see Database Report).
- **Result: PASSED (with performance warning)**

Scenario 5: Business Name & Trademark Guardrail Verification

- **Flow:** Student enters proposed business name (e.g., "Jollibee Food Hub") in `OfficialNameChecker.tsx` $\rightarrow$ Frontend invokes `POST /api/name-check` $\rightarrow$ Backend checks `NameCheckCache` $\rightarrow$ Runs `getTrademarkAlert()` against `copyright_db.json` $\rightarrow$ Flags conflict under RA 8293 / IPOPHL rules $\rightarrow$ Returns structured conflict alert $\rightarrow$ UI renders red warning banner blocking trademark infringement.
- **Observed Behavior:** Trademark conflict correctly prevents students from submitting infringing or confusingly similar well-known brand names.
- **Result: PASSED**

Scenario 6: Location Picker & Map Pin Persistence

- **Flow:** Student searches Philippine location (e.g., "Makati Avenue, Makati City") $\rightarrow$ `philippinePlaceSearch.ts` queries Nominatim with bounding box for the Philippines $\rightarrow$ Autocomplete suggestions populate $\rightarrow$ User selects location $\rightarrow$ Leaflet map centers and drops custom red pin $\rightarrow$ Clean address without raw GPS coordinates saves to `proposals.proposedLocation` $\rightarrow$ Reopening proposal restores pin and address.
- **Observed Behavior:** Address is correctly stored and reloaded. Note: Leaflet + OpenStreetMap is used rather than Google Maps API.
- **Result: PASSED**

Scenario 7: Unauthorized Access & Route Guarding

- **Flow:** Student attempts direct URL access to `/admin/users` $\rightarrow$ `ProtectedRoute.tsx` checks auth state and role $\rightarrow$ Recognizes "Student" role is not in `allowedRoles: ["Chairperson"]` $\rightarrow$ Redirects to `/dashboard` $\rightarrow$ No administrator data is exposed.
- **Observed Behavior:** Student is unable to access admin UI screens.

- **Security Limitation:** The backend API endpoint `PUT /api/proposals/:proposalId/status` does NOT enforce `verifyToken`, allowing unauthorized API callers to bypass frontend routing guards (documented in Security Report).
- **Result: PARTIALLY PASSED (Frontend protected, Backend endpoint exposed)**

Scenario 8: Network Interruption & Error Resilience

- **Flow:** Client attempts to submit proposal during temporary backend outage $\rightarrow$ Backend health check fails $\rightarrow$ Client falls back gracefully $\rightarrow$ Proposal data preserved in local state $\rightarrow$ User receives error toast without catastrophic crash.
- **Observed Behavior:** The application handles failed network promises with `try/catch` and displays user-friendly error banners.
- **Result: PASSED**

---

3. Real-Time Socket.IO & Notification Verification

Subsystem	Verified Behavior	Latency	Status
Socket.IO Connection	Client establishes handshake with backend (`http://localhost:10000`)	`< 45 ms`	**Passed**
Room Joining	Users join rooms based on `groupId`	`< 15 ms`	**Passed**
Chat Message Broadcast	Messages dispatched via socket reach peers in same group room	`< 80 ms`	**Passed**
Notification Service	`notificationService.ts` writes unread notifications to `notifications` collection	`< 120 ms`	**Passed**
Adviser Section Broadcast	`notifyAdvisersForSection` broadcasts notifications to all advisers handling that section	`< 150 ms`	**Passed**